Cloud Computing (CS3081) Mini Project Report

Amit Sharma (Roll No: 230103033) | Repository: https://github.com/amitishusharma2005-prog/cs3081-kd-command-parser

1. Object storage and parallel teacher labelling (Task 1)

Architecture. A k3d cluster (*cs3081-cluster*, config *cluster/k3d.yaml*, host port 8080 mapped to the load balancer) runs SeaweedFS in namespace *data* (S3 gateway on 8333, S3 identities from a generated Secret). A Job creates buckets *datasets* and *models* and uploads *commands_pool.jsonl*. The teacher is phi3.5 served by Ollama on the laptop (temperature 0, seed 42); pods reach it at *http://172.19.0.1:11434* (verified from inside a pod). **Prompt design** (*prompts/teacher_prompt.txt*): states the four keys and eight intents, the three slot rules, and nine few-shot examples taken from the assignment text; it demands a bare JSON object. **Sharding.** Two Jobs *worker-0/1* in namespace *pipeline* set SHARD_COUNT=2; worker k takes every line i (0-based) with i mod 2 = k. Each line has exactly one remainder, so every id lands in exactly one shard: no gaps, no repeats. Workers checkpoint to *datasets/teacher/shard-k.jsonl* and resume after a restart.

Cleaning (*worker/clean.py*): records sorted by id, then each gets one verdict: (a) invalid JSON / not an object, (b) wrong keys, (c) invalid intent, (d) slot not null and not a substring of the command, then duplicate command text (smallest id kept). Output is deterministic, so re-running yields the same checksum.

Input	(a)	(b)	(c)	(d)	Duplicates	Kept
3000	1	0	10	79	47	2863

input = a+b+c+d+duplicates+kept: 3000 = 3000. train.sha256 = f1dd2333d216aca933a4217d532493ac372526aea8f3dd79529aaa190dce79bf

2. Training the student (Task 2)

Setup. *training/config.yaml* holds seed 42, learning rate 0.0002, batch size 8, 8 epochs, max length 128 and library versions; v2 uses seed 43 and 8 epochs. Examples are *Command: <text>\nJSON: <teacher JSON>*. Tokens up to and including 'JSON:' are labelled -100, so cross-entropy covers only the JSON reply and the EOS token. Seeds are fixed for random, numpy, torch and transformers; AdamW with linear warm-up/decay, gradient clipping at 1.0, first 1,500 cleaned records. Model + tokenizer go to *s3://models/v1/* and a deterministic tar.gz is the GitHub Release asset.

model.sha256 v1 = 0e9c66d4a4d7a4147dc87aadd27ace07a0f4c749569f37cc2b505a317d2a756a; v2 = 3c51155780c6de0f6bd5e0bf5728ac535d9820da80db8d5fef73ee1c7c46b98d

epoch	train_loss
1	0.5298
2	0.3320
3	0.1551
4	0.0988
5	0.0632
6	0.0347
7	0.0182
8	0.0073

Metric (%)	Teacher phi3.5	Untrained Pythia-70M	Student v1	Student v2
json_validity	99.0	0.0	100.0	100.0
intent_accuracy	87.0	0.0	88.0	83.0
date_accuracy	78.0	0.0	87.0	82.0
time_accuracy	85.0	0.0	90.0	88.0
place_accuracy	85.0	0.0	80.0	83.0
full_frame	48.0	0.0	53.0	48.0

Discussion. Write after the numbers exist: compare students with the untrained base and with the teacher, and say whether full_frame is at least half of the teacher's.

Sequence-level vs logit-based distillation

(a) **Sequence-level KD** trains the student with ordinary cross-entropy on the teacher's generated output sequences (hard targets) as if they were labels. (b) **Logit-based KD** trains the student to match the teacher's per-token output probability distribution (e.g. KL divergence over the vocabulary), giving soft targets that carry the teacher's uncertainty. (c) It is impossible here because Phi-3.5 and Pythia-70M use different tokenizers and vocabularies, so their distributions cannot be aligned token by token, and Ollama's API returns only generated text, not full logits. (d) **Limitation:** the student only sees one sampled answer per command, so it inherits every teacher mistake that survives cleaning (a wrong but substring-valid slot passes rule (d)) and gets no signal about the teacher's confidence.

3. Inference service in a container (Task 3)

service/app.py (FastAPI, port 8000) serves POST /parse, GET /healthz (200 while the process lives) and GET /readyz (503 until the model is loaded, then 200). Configuration comes only from MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS (plus FAULT_INJECT). At start-up a background thread downloads the model/tokenizer from object storage using credentials from a Kubernetes Secret, so the process is live but not ready while loading. Every output is validated

against *data/schema.json* (JSON object, exactly four keys, valid intent, slots null or string); failures return HTTP 500 *{"error": "invalid_model_output"}*. FAULT_INJECT=1 replaces the output with *not json* before validation. Success returns only the four keys plus X-Model-Version and X-Served-By (pod name). The image uses python:3.11-slim, CPU-only torch wheels and pinned direct dependencies, and contains no weights.

Measured: image size 1615.7 MB (limit 2048 MB); 50 sequential requests at --cpus=1.0 --memory=1g: 50/50 returned HTTP 200, OOMKilled=false, peak memory 990.1 MiB (cgroup memory.peak) / 480.4 MiB highest docker stats sample. Evidence: results/docker_50req_limit_test.log.

4. Kubernetes deployment, scaling and reliability (Task 4)

cluster/app-deployment.yaml (namespace *app*): Deployment with 2 replicas, RollingUpdate maxUnavailable 0 / maxSurge 1, requests 250m/512Mi, limits 1 CPU/1Gi, liveness on /healthz, readiness on /readyz, S3 credentials via secretKeyRef; Service on 8000; Traefik Ingress sending /parse to the Service; HPA min 1, max 2, CPU target 70%. Because readiness depends on /readyz, a Pod receives no traffic (is absent from the Service endpoints) until the model is loaded.

Limitation of the local cluster: the cluster is a single k3d node on one laptop, so it demonstrates scheduling, self-healing and rolling updates, but **not** fault tolerance across machines: if the laptop or Docker daemon fails, every replica fails together. Metrics-server on k3s also reacts slowly, so HPA scale-up/down is delayed.

Measured (logs in results/, produced by scripts/evidence.sh): readiness gate: Service endpoints showed <none> while the model loaded = True, endpoints appeared afterwards = True. HPA: scaled 1→2 = False, back to 1 = True. Rolling update v1→v2 under load: 2.151% failed of 279 requests (limit 0.5%); versions served {'v1': 129, 'v2': 146}; after *kubectl rollout undo* X-Model-Version = v1. Pod deletion under load: 0.0% failed (limit 2%), Ready Pods at end = 2. Cold start per run (s): {'1': 23, '2': 59, '3': 58}, **median 58 s** (per run = slowest new Pod, creationTimestamp -> Ready; median over runs); the logs show the S3 download lines.

Assumptions: the Ingress is reached through k3d's port 8080; HPA reacts at metrics-server's resolution; replicas run on the same node.

Measured load balancing (two Ready replicas, standard Locust run): per-replica successful requests {"command-parser-55d4dccb84-p4ngr": 447, "command-parser-55d4dccb84-6wj94": 397} = {"command-parser-55d4dccb84-p4ngr": 52.96, "command-parser-55d4dccb84-6wj94": 47.04} % (each must be at least 30%); model versions: {"v1": 844}.

5. Subnetting and network access control (Task 5)

Subnet	Needed	Network	Prefix	Usable range	Broadcast	Usable
app	1000	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1022
public	250	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Working: smallest prefix with 2^(32-p) - 2 >= hosts: 1000 → /22 (1022; /23 gives 510); 250 → /24 (254); 100 → /25 (126); 20 → /27 (30). Blocks placed back to back from 10.20.0.0.

Dir	Rule	Action	Protocol/ports	CIDR
in	100	deny	all	198.51.100.0/24
in	110	allow	tcp 80	0.0.0.0/0
in	120	allow	tcp 443	0.0.0.0/0
in	130	allow	tcp 22	10.20.5.128/27 (management)
in	140	allow	tcp 1024-65535 (replies)	10.20.0.0/22 (app)
in	32000	deny	all	0.0.0.0/0
out	100	allow	tcp 8000	10.20.0.0/22 (app)
out	110	allow	tcp 1024-65535 (replies)	0.0.0.0/0
out	32000	deny	all	0.0.0.0/0

Measured: *nacl_sim.py --check* on the sample packets, output in results/nacl_check.log; tests/test_all.py adds 15 further packets.

A NACL is **stateless**: it keeps no connection table, so an allowed request does not make its reply allowed. Replies go to the client's ephemeral port (1024-65535), so each direction needs its own explicit reply rule. Rules are checked lowest number first and the first match wins, so the deny for 198.51.100.0/24 must be numbered below the allows for TCP 80/443; otherwise a packet from that range to port 443 would match the allow first.

6. Reproducibility, testing and CI/CD (Task 6)

make up builds and deploys everything from a fresh clone (cluster, generated secrets, SeaweedFS, models fetched from the GitHub Release and checksum-verified, image, manifests); *make down* deletes the cluster; *make train [SUBSET=50] [VERSION=v2]* trains; *make test* runs 30 unit tests plus the NACL check. The GitHub Actions workflow runs: lint (ruff) → unit tests

→ model evaluation (Release asset, checksum verified, score.py on validation.jsonl) → quality gate (score.py --min-validity 95) → gitleaks. A manual run with candidate=untrained shows the gate failing.

Measured: unit tests passed = 33; release asset checksums match models/v*/model.sha256 = True; gitleaks reported no leaks = True. Failing quality-gate run (untrained Pythia-70M): https://github.com/amitishusharma2005-prog/cs3081-kd-command-parser/actions/runs/42536975945. Passing run (student from the Release): https://github.com/amitishusharma2005-prog/cs3081-kd-command-parser/actions/runs/42537234193. AI_USE.md documents the AI tooling and verification.

7. Performance, cost and review (Task 7)

Requests	Failures	Error rate	p50 (ms)	p95 (ms)	Req/s
320	0	0.00%	2000	2400	2.687700887365216
Pod		CPU (kubectl top, during load)		Memory	
command-parser-55d4dccb84-p4ngr		1000m		521Mi	
command-parser-55d4dccb84-ttgfp		998m		469Mi	

Cold start: see Section 4. **Assumptions:** the rate card is hypothetical; replicas are billed for CPU/memory *limits* for the whole hour; throughput measured over one 2-minute run is assumed to hold for an hour.

Cost (rate card: INR 3.50 per vCPU-hour, 0.45 per GiB-hour): per replica = 1 x 3.50 + 1 x 0.45 = 3.95; all running replicas = 7.90 per hour; requests per hour = 2.687700887365216 x 3600 = 9676; cost per 1,000 requests = hourly / requests per hour x 1000 = **0.8165 INR** (estimate, no real charge).

SLO: >= 99.0% of requests return HTTP 200 within 30 s over one 2-minute load-test run. Error budget = 320 x 1% = 3.2 requests; used 0 (0.0%).

Component	Evidence (measured number)	Limitation	Improvement
Data and storage	2863 clean records kept	Single SeaweedFS pod, no replication; teacher errors that pass the substring check stay in the data	Replicated volumes; second-pass validation of teacher labels
Model quality	full_frame 53.0%	Sequence-level KD copies teacher mistakes; 70M model	Larger student or more/cleaner data
Latency and throughput	p95 2400 ms	One torch thread per replica; no batching	Dynamic batching, ONNX/quantisation
Resource utilisation	see kubectl top log	Requests/limits chosen by estimate	Tune from measured usage
Reliability and recovery	see recovery log	Single node: no machine-level fault tolerance	Multi-node cluster, PodDisruptionBudget
Security and cost	0.8165 per 1,000 requests	Static generated S3 keys, no TLS, replicas billed even when idle	Short-lived credentials, TLS ingress, scale to demand

References

Kim & Rush (2016) Sequence-Level Knowledge Distillation, EMNLP. Bastianelli et al. (2020) SLURP, EMNLP. Biderman et al. Pythia model card. k3d, SeaweedFS, Kubernetes, Locust, Gitleaks documentation; AWS VPC network ACL guide; Google SRE Book (SLOs).